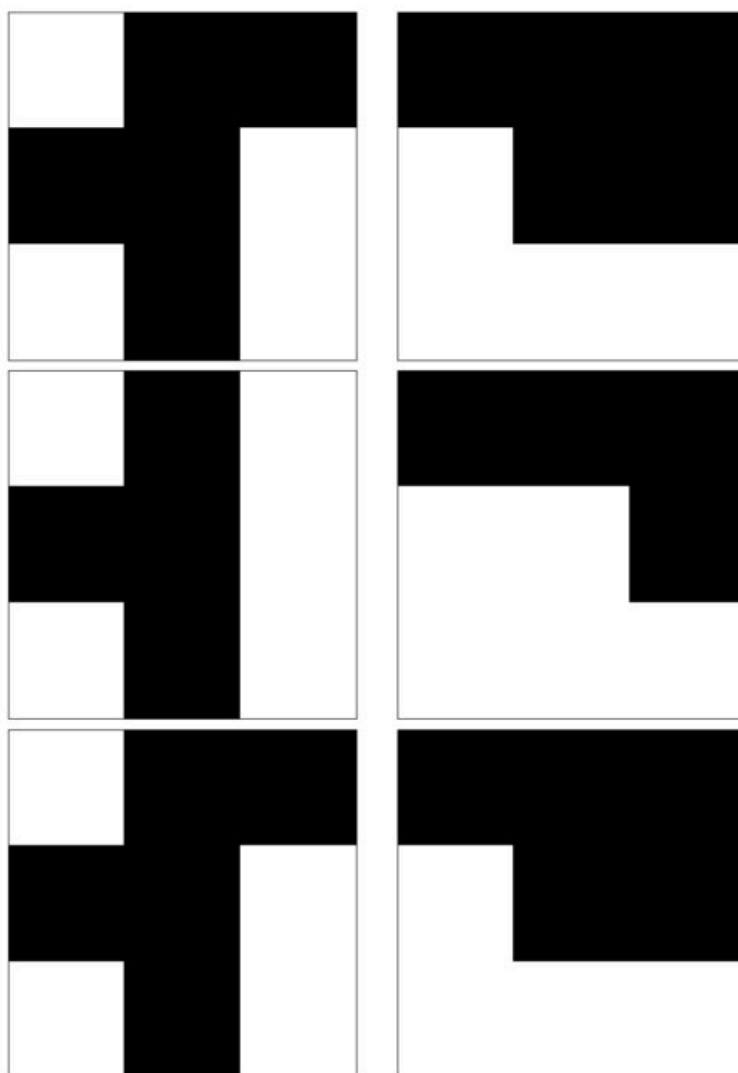


Понимание обучения по принципу Хебба в сетях Хопфилда

✍ 3 марта 2024 г. • ⌚ 6 минут чтения • 👁 см. также • 💬 ветка (<https://sigmoid.social/@pixeltracker/112035901352203372>) • 💬 комментарии

Сети Хопфилда, разновидность рекуррентных нейронных сетей (РНС), служат фундаментальной моделью для изучения ассоциативной памяти и распознавания образов в компьютерной нейронауке. В основе работы сетей Хопфилда лежит правило обучения Хебба — идея, выраженная в принципе «нейроны, которые возбуждаются одновременно, связываются друг с другом». В этом посте мы рассмотрим математические основы обучения Хебба в сетях Хопфилда, уделив особое внимание его роли в распознавании образов.



Сети Хопфилда

Сеть Хопфилда состоит из набора нейронов, которые могут находиться либо во включенном (+1), либо в выключенном (-1) состоянии (такие нейроны также называют бинарными нейронами Маккалока — Питтса). Сеть относится к классу сетей с обратной связью и состоит всего из одного слоя, который одновременно является входным и выходным. Сеть полносвязная, то есть каждый нейрон связан с каждым другим нейроном определенным весом, кроме самого себя. Эти веса определяют динамику работы сети и корректируются в соответствии с правилом обучения Хебба для сохранения определенных паттернов.

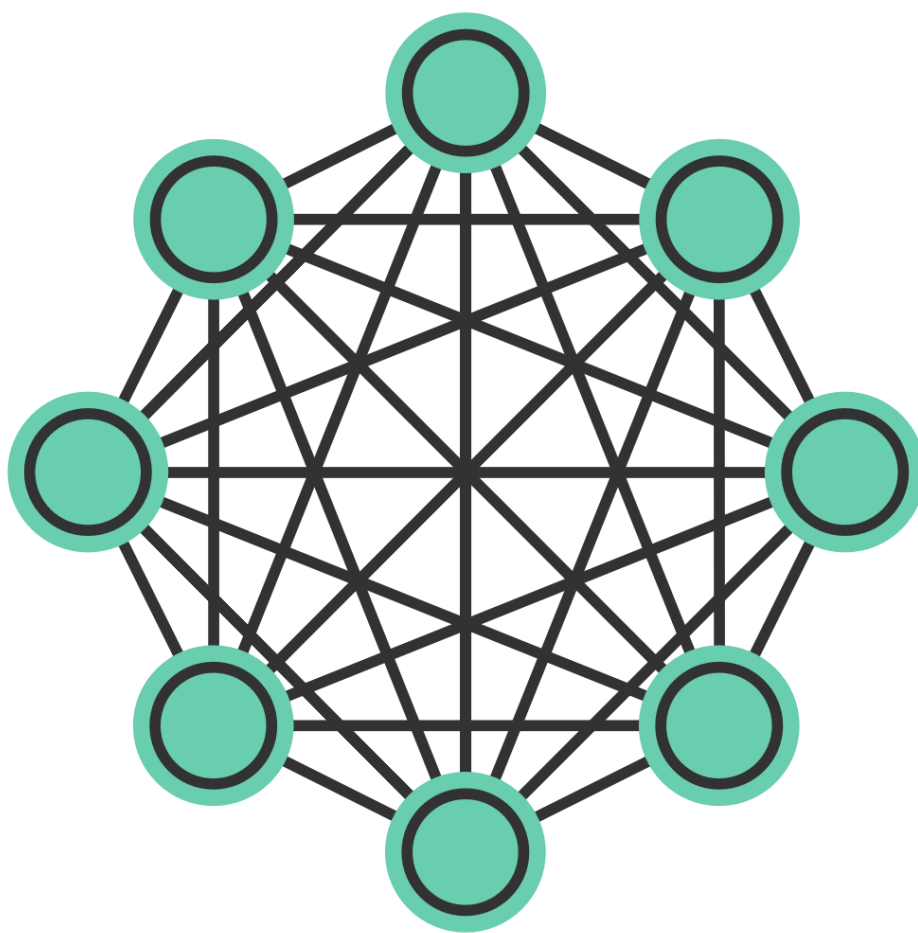


Схема сети Хопфилда. Все нейроны связаны друг с другом с определенным весом, кроме самих себя. Так называемые нейроны Маккалока — Питтса могут находиться в состоянии +1 или -1.

Математическая основа

Вес w_{ij} между двумя нейронами i и j в сети Хопфилда обновляется на основе принципа обучения Хебба следующим образом:

$$w_{ij} = \frac{1}{N} \sum_{\mu=1}^P \xi_i^{\mu} \xi_j^{\mu}$$

где:

- N — общее количество нейронов в сети,
- P — количество шаблонов, которые нужно запомнить,
- ξ_i^{μ} — это состояние (-1 или 1) нейрона i в шаблоне μ .

Диагональные элементы весовой матрицы равны нулю, $w_{ii} = 0$ чтобы нейроны не влияли сами на себя (так как самосвязи недопустимы). В бинарном паттерне, состоящем только из единиц и -1, ξ_i^{μ} соответствует значению i -й элемент μ -й шаблон. Таким образом, мы можем установить $\xi_i^{\mu} = 1$ если i -й элемент μ -й паттерн равен 1, а $\xi_i^{\mu} = -1$ если i -й элемент μ -й паттерн равен -1, то есть $\xi_i^{\mu} = \text{pattern}^{\mu}$.

Приведенная выше формула гарантирует, что весовые коэффициенты будут скорректированы таким образом, чтобы усилить корреляцию между нейронами, которые одновременно активны в сохраненных паттернах. Процесс обучения включает в себя корректировку весовых коэффициентов на основе сохраненных паттернов. Эта операция эффективно реализует правило обучения Хебба, усиливая связь между нейронами, которые одновременно активны в одном и том же паттерне.

После обучения сеть Хопфилда может распознавать паттерны, переходя из заданного начального состояния (которое может представлять собой зашумленную или неполную версию сохраненного паттерна) в стабильное состояние, соответствующее одному из запомненных паттернов. Математически этот процесс описывается следующим образом:

$$s_i^{(t+1)} = \text{sgn} \left(\sum_j w_{ij} s_j^{(t)} \right)$$

где $s_i^{(t+1)}$ — это состояние нейрона i в момент времени $t + 1$, и $\text{sgn}(\cdot)$ Это знаковая функция, которая обеспечивает обновление состояния нейрона до +1 или -1 в зависимости от взвешенной суммы входных данных.

Application to pattern recognition

To demonstrate the application of Hebbian learning in pattern recognition using Hopfield networks, let's start by implementing a simple Hopfield network class in Python:

```
class HopfieldNetwork:
    def __init__(self, size):
        self.size = size
        self.weights = np.zeros((size, size))
```

Next, we add a method to train the network using the Hebbian learning rule:

```
def train(self, patterns):
    for pattern in patterns:
        pattern = np.reshape(pattern, (self.size, 1)) # reshape to a column vector
        self.weights += np.dot(pattern, pattern.T) # update weights based on Hebbian
learning rule, i.e., weights = weights + pattern*pattern'
        self.weights[np.diag_indices(self.size)] = 0 # set diagonal to 0 in order to avoid
self-connections
        self.weights /= self.size # normalize weights by the number of neurons to ensure
stability of th network
```

In this training function, we iterate over the provided patterns and update the weights based on the Hebbian learning rule. We then normalize the weights by the number of neurons to ensure the stability of the network.

Finally, we add a method to predict the network's output based on a given input:

```
def predict(self, pattern, steps=10):
    pattern = pattern.copy()
    for _ in range(steps):
        for i in range(self.size):
            raw_value = np.dot(self.weights[i, :], pattern)
            pattern[i] = 1 if raw_value >= 0 else -1
    return pattern
```

The prediction function illustrates how a Hopfield network can recover stored patterns from corrupted inputs. It updates each neuron's state based on the current pattern and the learned weights, iteratively moving the network towards a stable state that corresponds to one of the memorized patterns.

That's almost everything we need to implement a simple Hopfield network in Python. We can now use this class to train the network on a set of patterns and then test its ability to recover these patterns from noisy or incomplete inputs. Below is the complete Python code for the Hopfield network class and an example of its usage:

```

import numpy as np
import matplotlib.pyplot as plt

# define the Hopfield network class:
class HopfieldNetwork:
    def __init__(self, size):
        self.size = size
        self.weights = np.zeros((size, size))

    def train(self, patterns):
        for pattern in patterns:
            pattern = np.reshape(pattern, (self.size, 1)) # reshape to a column vector
            self.weights += np.dot(pattern, pattern.T) # update weights based on Hebbian
learning rule, i.e.,  $W = W + p \cdot p^T$ 
            self.weights[np.diag_indices(self.size)] = 0 # set diagonal to 0 in order to avoid
self-connections
            self.weights /= self.size # normalize weights by the number of neurons to ensure
stability of th network

    def predict(self, pattern, steps=10):
        pattern = pattern.copy()
        for _ in range(steps):
            for i in range(self.size):
                raw_value = np.dot(self.weights[i, :], pattern)
                pattern[i] = 1 if raw_value >= 0 else -1
        return pattern

    def visualize_patterns(self, patterns, title):
        fig, ax = plt.subplots(1, len(patterns), figsize=(10, 5))
        for i, pattern in enumerate(patterns):
            ax[i].matshow(pattern.reshape((int(np.sqrt(self.size)), -1)), cmap='binary')
            ax[i].set_xticks([])
            ax[i].set_yticks([])
        fig.suptitle(title)
        plt.show()

# define test patterns
pattern1 = np.array([-1,1,1,1,1,-1,-1,1,-1]) # representing a simple shape
pattern2 = np.array([1,1,1,-1,1,1,-1,-1,-1]) # another simple shape
patterns = [pattern1, pattern2]

# initialize Hopfield network:
network_size = 9 # this should be square to easily visualize patterns
hn = HopfieldNetwork(network_size)

# train the network:
hn.train(patterns)

# corrupt patterns slightly:
corrupted_pattern1 = np.array([-1,1,-1,1,1,-1,-1,1,-1]) # slightly modified version of ↱
pattern1

```

```

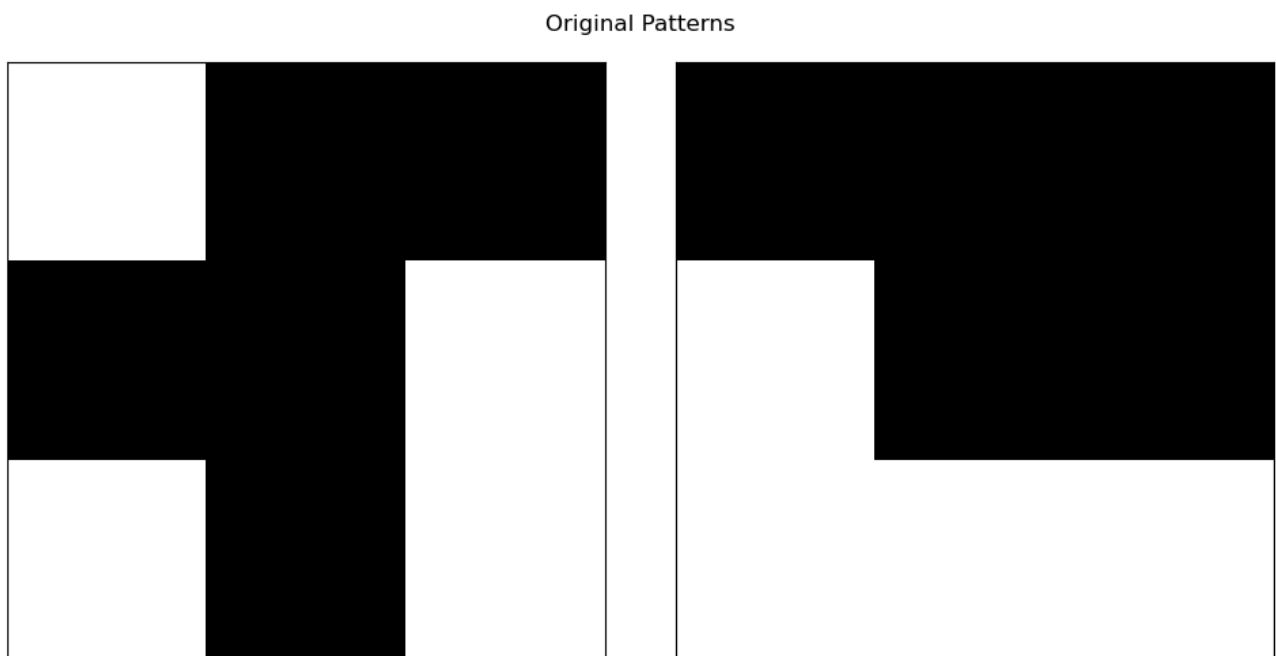
corrupted_pattern2 = np.array([1,1,1,-1,-1,1,-1,-1,-1]) # slightly modified version of
pattern2
corrupted_patterns = [corrupted_pattern1, corrupted_pattern2]

# predict (recover) from corrupted patterns:
recovered_patterns = [hn.predict(p) for p in corrupted_patterns]

# visualize original, corrupted, and recovered patterns:
hn.visualize_patterns(patterns, 'Original Patterns')
hn.visualize_patterns(corrupted_patterns, 'Corrupted Patterns')
hn.visualize_patterns(recovered_patterns, 'Recovered Patterns')

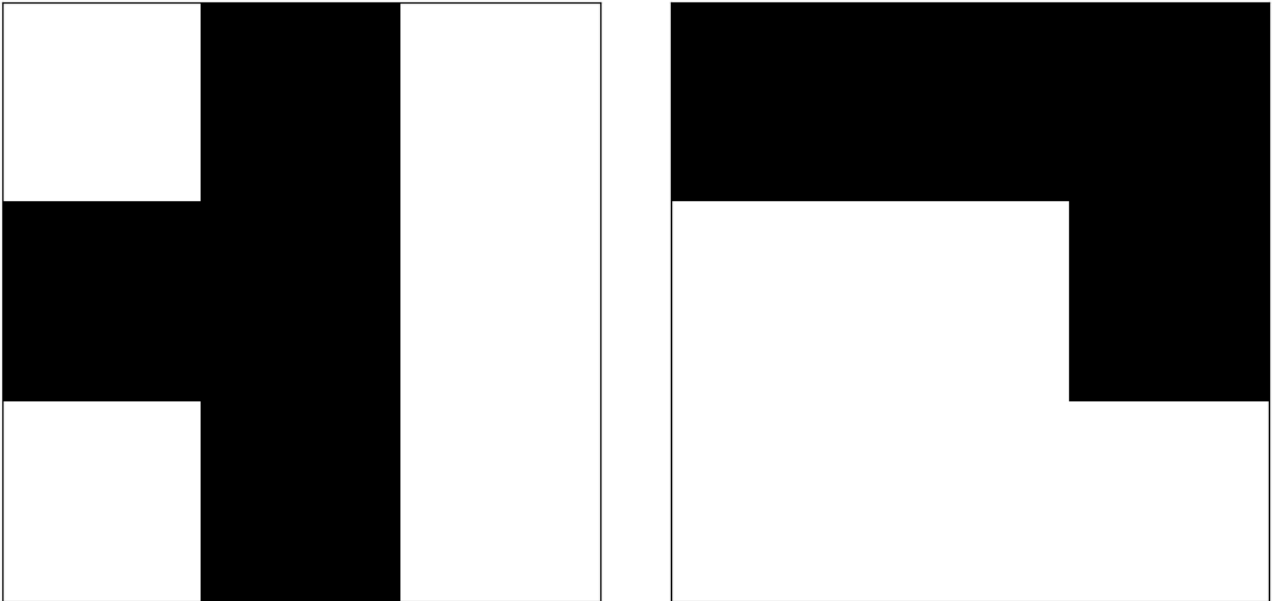
```

Note, that I added an additional `visualize_patterns` method to the `HopfieldNetwork` class to visualize the patterns. We define two patterns,



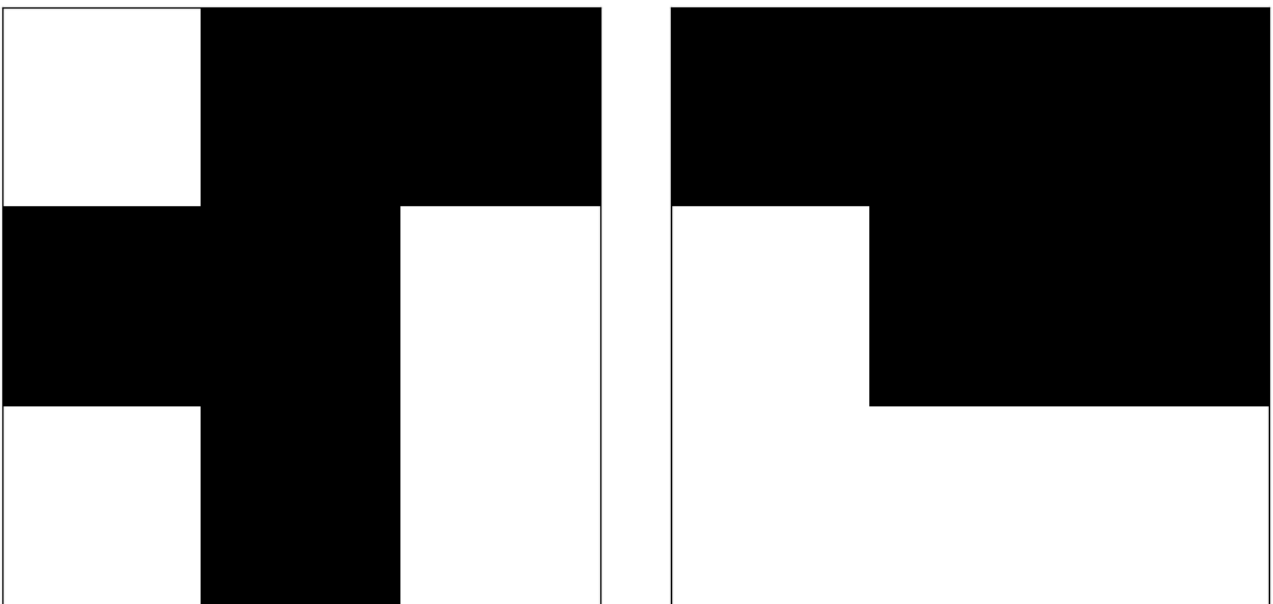
and train the network on these patterns. We then slightly corrupt the patterns,

Corrupted Patterns



and use the trained network to recover the original patterns from the corrupted versions:

Recovered Patterns



From the recovered patterns, we can see that the Hopfield network successfully recovered the original patterns from the corrupted versions, demonstrating that the network has learned to recognize and recall the memorized patterns.

Conclusion

Hebbian learning forms the core of Hopfield networks, enabling them to store and recognize patterns. Through iterative adjustments of neuronal connections based on the principle that [↑]“neurons that fire together, wire together”, Hopfield networks can recover original patterns from

noisy or incomplete versions, showcasing their utility in pattern recognition tasks. The mathematical formulation and Python implementation presented here hopefully offer a foundational understanding of how Hopfield networks operate. You can play around with the code to train the network on different patterns and observe its ability to recover them from corrupted inputs. Adding noise to the patterns and testing the network's robustness is also an interesting experiment to try.

You can find the complete code for this post also in this GitHub repository

(https://github.com/FabrizioMusacchio/hopfield_network)[†].

References

- Hopfield, John J., *Neural networks and physical systems with emergent collective computational abilities*, 1982, Proc Natl Acad Sci U S A, 79(8), 2554-2558. doi: 10.1073/pnas.79.8.2554
- Hopfield, John J., *Neurons with graded response have collective computational properties like those of two-state neurons*, 1984, Proc Natl Acad Sci U S A, 81(10), 3088-3092. doi: 10.1073/pnas.81.10.3088
- Hebb, Donald O., *The Organization of Behavior*, 1949, Wiley: New York, pages 437, url, doi: 10.1016/s0361-9230(99)00182-3
- Wulfram Gerstner, Werner M. Kistler, Richard Naud, and Liam Paninski, *Neuronal Dynamics: From Single Neurons to Networks and Models of Cognition*, 2014, Cambridge University Press, <https://www.cambridge.org/9781107060838>, Online-Version

🔖 **Теги:**

Вычислительная наука

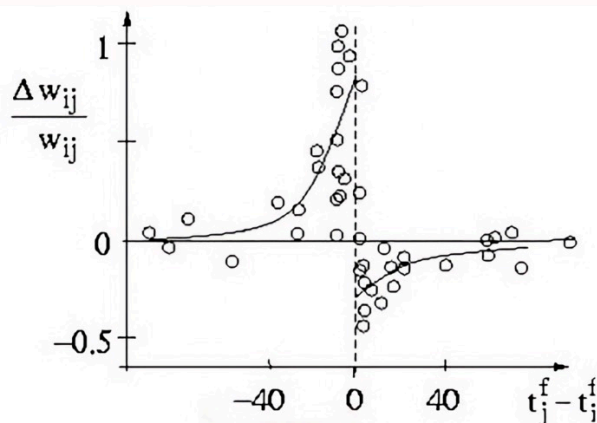
Наука о данных

Нейронаука

Python

📅 **обновлено:** 3 марта 2024 г.

7 OTHER ARTICLES ARE LINKED TO THIS SITE

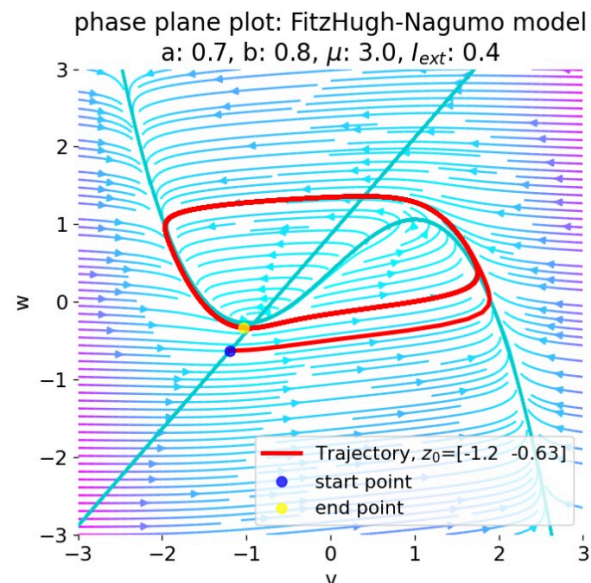


Spike-timing-dependent plasticity (STDP)

February 12, 2026

🕒 25 minute read

Another frequently used term in computational neuroscience is spike-timing-dependent plasticity or STDP. STDP is a form of...

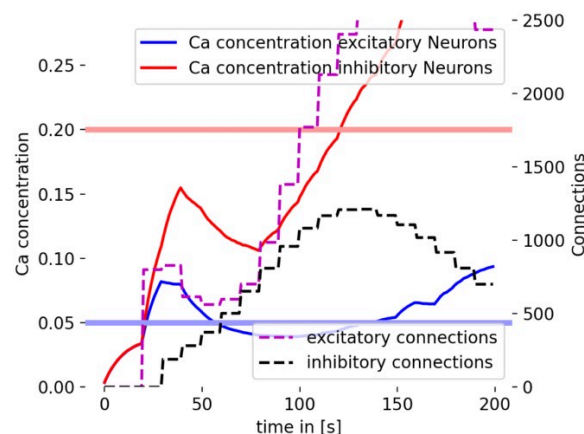
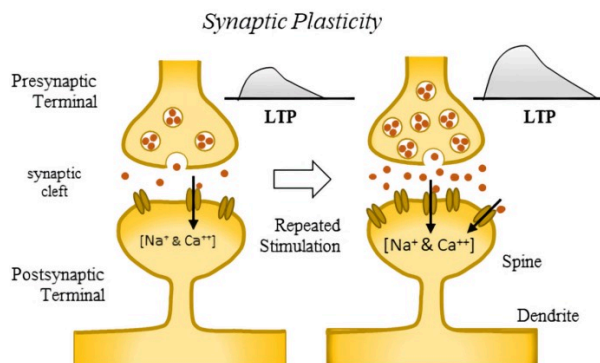


Neural Dynamics: A definitional perspective

February 4, 2026

🕒 23 minute read

Neural dynamics is a subfield of computational neuroscience that focuses on the time dependent evolution of neural activit...



Neural plasticity and learning: A computational perspective

February 2, 2026

🕒 30 minute read

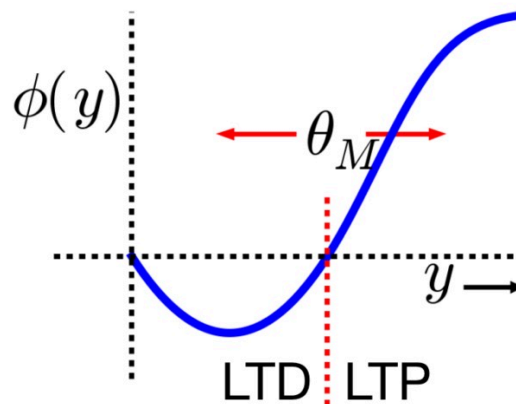
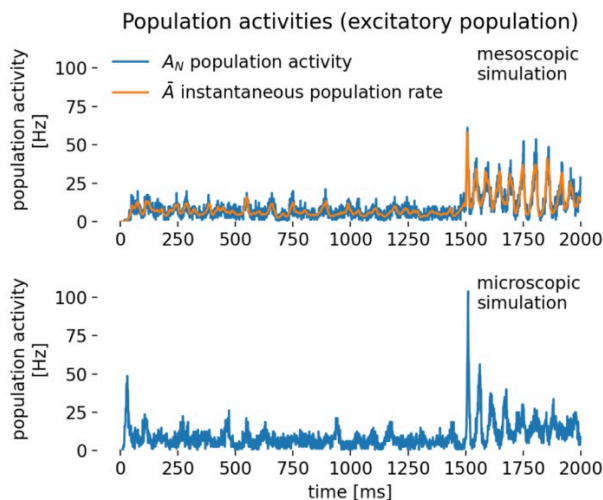
After discussing structural plasticity in the previous post, we now take a broader look at neural plasticity and learning ...

Incorporating structural plasticity in neural network models

February 1, 2026

🕒 12 minute read

In standard spiking neural networks (SNN), synaptic connections between neurons are typically fixed or change only accordi...



Rate models as a tool for studying collective neural activity

August 28, 2025

🕒 20 minute read

Rate models provide simplified representations of neural activity in which the precise spike timing of individual neurons

...

Bienenstock-Cooper-Munro (BCM) rule

September 8, 2024

🕒 15 minute read

The Bienenstock-Cooper-Munro (BCM) rule is a cornerstone in theoretical neuroscience, offering a comprehensive framework f...



NEST simulator – A powerful tool for simulating large-scale spiking neural networks

June 9, 2024

🕒 6 minute read

updated: June 12, 2024

The NEST simulator is a powerful software tool designed for simulating large-scale networks of spiking neurons (SNN). It h...

COMMENTS